EX25 · NumPy for linear algebra

Expanded reference: inputs, outputs, executable examples, common mistakes and automated testing.

LANGUAGE / ID	UNIT	HANDOUT
EN · EX25	algebra / vectors and linear systems	Python Reference Guide

1 · Mental model

A Python list stores values; a NumPy **ndarray** represents numerical data with explicit dimensions. A 2D vector is generally shaped **(2,)**, whereas a two-row, two-column matrix has shape **(2, 2)**. Shape matters as much as the numerical values.

```
import numpy as np
u = np.array([2.0, 3.0], dtype=float)
A = np.array([[3.0, 1.0], [1.0, 2.0]], dtype=float)

print(u.shape)     # (2,)
print(A.shape)     # (2, 2)
print(u.ndim)      # 1
print(A.ndim)      # 2
```

Concept	Example	Meaning
shape	A.shape → (2, 2)	Number of rows and columns
ndim	u.ndim → 1	Number of array dimensions
dtype	u.dtype	Stored numerical type
axis	axis=0 / axis=1	Direction of array computation

2 · Different multiplication operations

```
A = np.array([[3., 1.], [1., 2.]])
u = np.array([2., 3.])
print(A @ u)  # [9., 8.] -> matrix-vector product
print(A * u)  # [[6., 3.], [2., 6.]] -> broadcasting
print(u * u)  # [4., 9.] -> element-wise product
print(np.dot(u, u))  # 13.0 -> dot product
```

Rule of thumb: A @ u applies the linear transformation represented by A. A * u multiplies array elements using broadcasting, which does not model the required resultant force. Use @ when the physics describes a linear combination.

3 · np.linalg.solve(A, b): solve A @ u = b

Inputs: square matrix A and a compatible target vector b. **Output:** solution vector u, provided A is nonsingular. **When to use:** simultaneous linear equations with as many unknowns as equations.

```
A = np.array([[3., 1.], [1., 2.]])
b = np.array([9., 8.])
u = np.linalg.solve(A, b)
print(u)       # [2., 3.]
print(A @ u)   # [9., 8.]
```

> Avoid routinely using **np.linalg.inv(A) @ b** to solve systems. **np.linalg.solve** does not explicitly construct the inverse and is typically more stable and efficient.

4 · Norms and residuals

np.linalg.norm(x) returns the Euclidean norm (length) by default for a vector. The residual measures the difference between the reconstructed resultant and target force.

$$\mathbf{r} = \mathbf{Au} - \mathbf{b}$$

$$\text{residual}_{abs} = \|\mathbf{r}\|_2$$

```
u = np.array([2., 3.])
b = np.array([9., 8.])
residual_vector = A @ u - b
absolute_error = float(np.linalg.norm(residual_vector))
relative_error = absolute_error / float(np.linalg.norm(b))
print(absolute_error)  # 0.0 (or nearly zero)
```

Important: when ||b|| = 0, the stated relative residual is undefined. This exercise requires a **ValueError**; do not silently add a small number to the denominator.

5 · np.linalg.cond: sensitivity

Inputs: matrix A, optionally the norm order **p=2**. **Output:** a dimensionless scalar measuring how strongly relative data perturbations can affect the solution.

```
print(np.linalg.cond(np.eye(2), 2))  # 1.0
B = np.array([[1., 1.], [1., 1.000001]])
print(np.linalg.cond(B, 2))       # approximately 4e6
```

A high condition number signals sensitivity. It is not the residual: one result may reconstruct b almost perfectly yet change dramatically after a tiny perturbation of b.

6 · Input validation and errors

Engineering functions should reject incompatible inputs. Use **np.asarray(..., dtype=float)** to accept lists and convert them to arrays; check shapes using **.shape** and finite values using **np.isfinite**.

```
def check_inputs(A, b):
    A = np.asarray(A, dtype=float)
    b = np.asarray(b, dtype=float)
    if A.shape != (2, 2) or b.shape != (2,):
        raise ValueError("Expected A (2,2) and b (2,)")
    if not np.isfinite(A).all() or not np.isfinite(b).all():
        raise ValueError("All values must be finite")
    return A, b
```

This is a **reference example** to adapt, not an implementation of the complete exercise. **np.linalg.solve** raises **np.linalg.LinAlgError** when the matrix is singular.

7 · Compare floating-point results safely

Floating-point numbers approximate binary fractions, so **0.1 + 0.2 == 0.3** can be False. Use tolerances: **np.isclose** for scalars and **np.allclose** for arrays.

```
np.isclose(0.1 + 0.2, 0.3)   # True
np.allclose([2., 3.], [2.00000000001, 3.])  # True
np.allclose(A @ u, b, atol=1e-10, rtol=0)   # check tolerance
```

The **atol** and **rtol** parameters are absolute and relative tolerance. Setting **rtol=0** makes the check use an absolute tolerance in physical units.

8 · Dictionaries and return values

A dictionary provides names for different physical quantities. For sensitivity output, use arrays for vectors and Python numbers for scalar quantities; call **float(...)** on numerical norms if necessary.

```
report = {
    "u_original": np.array([2., 3.]),
    "u_perturbed": np.array([2.1, 2.9]),
    "delta_u": np.array([0.1, -0.1]),
    "delta_u_norm": float(np.linalg.norm([0.1, -0.1])),
    # ... remaining fields ...
}
```

9 · pytest and incremental debugging

```
python -m pytest -q
python -m pytest -q -x
python -m pytest tests/test_algebra.py -vv
python main.py
```

Use **-x** to stop on the first failed test; **-vv** shows individual test names. Distinguish NotImplementedError (work left to do), ValueError (invalid input), and LinAlgError (singular system).

10 · Quick reference

Your goal	Use
Construct vector or matrix	np.asarray(..., dtype=float) / np.array(...)
Check dimensions	array.shape
Multiply matrix × vector	A @ u
Solve linear equations	np.linalg.solve(A, b)
Euclidean norm	np.linalg.norm(x)
Condition number	np.linalg.cond(A, 2)
Compare numeric results	np.allclose / np.isclose
Reject NaN and ±Inf	np.isfinite(...).all()